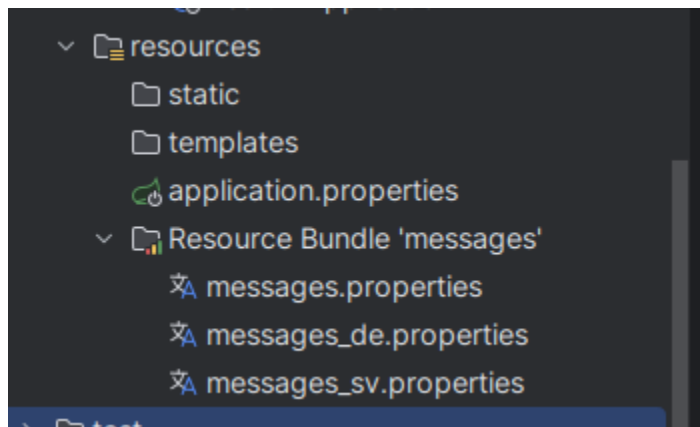


Assignment - RestFul Web Services - Part 2

Q1)Internationalization

A) Add support for Internationalization in your application allowing messages to be shown in English, German and Swedish, keeping English as default.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-hateoas</artifactId>
</dependency>
```



```
greeting.message=Hello {0}
```

```
greeting.message=Hallo {0}
```

```
greeting.message=Hej {0}
```

B) Create a GET request which takes "username" as param and shows a localized message "Hello Username". (Use parameters in message properties)

```
import org.springframework.context.MessageSource;
import org.springframework.web.bind.annotation.*;

import java.util.Locale;

@RestController
public class GreetingController {

    @Autowired
    private MessageSource messageSource;

    @GetMapping("/greet")
    public String greet(@RequestParam String username, @RequestHeader(name = "Accept-Language", required = false) Locale locale) {
        return messageSource.getMessage( code: "greeting.message", new Object[]{username}, locale);
    }
}
```

The screenshot shows the Postman interface with a new GET request configured. The URL is `http://localhost:8080/greet?username=Akash`. The Headers tab is active, showing a table with one header: `Accept-Language` with value `sv`. The response is a `200 OK` with a response time of `4 ms` and a body of `Hej Akash`.

Key	Value	Description
☒ Accept-Language	sv	
Key	Value	Description

Body: 1 Hej Akash

```
import java.util.Locale;

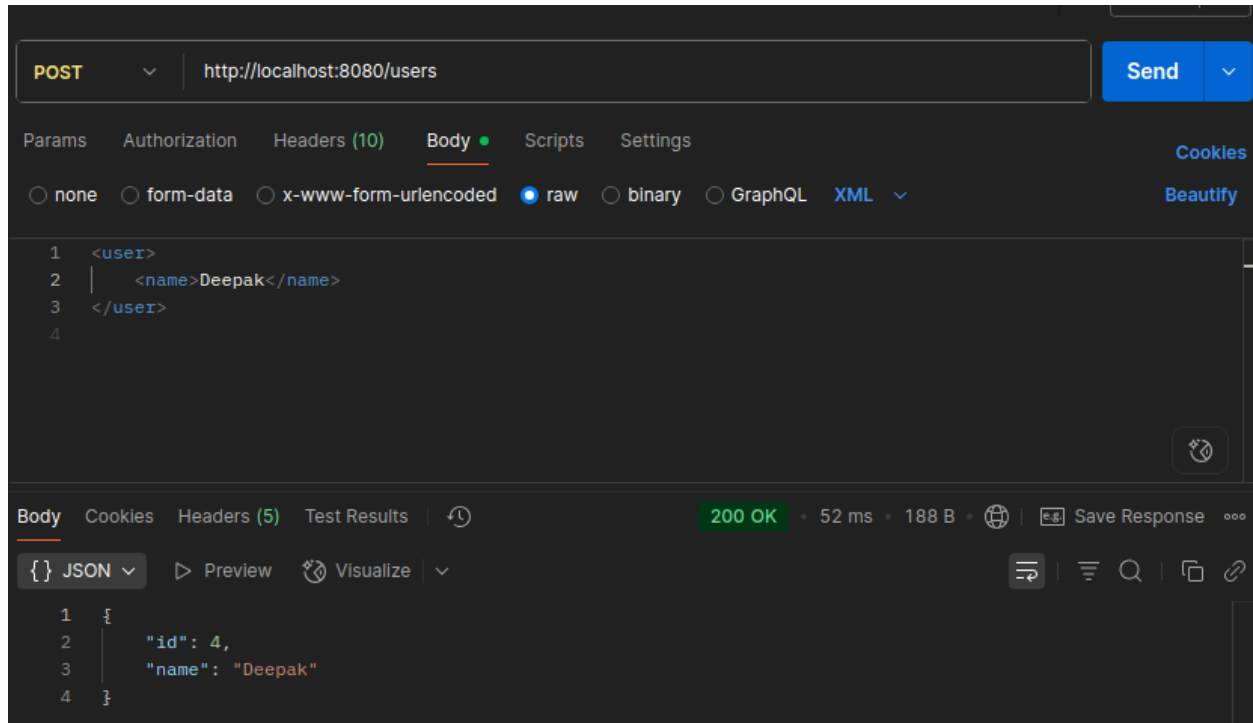
@Configuration
public class I18nConfig {

    @Bean
    public LocaleResolver localeResolver() {
        AcceptHeaderLocaleResolver resolver = new AcceptHeaderLocaleResolver();
        resolver.setDefaultLocale(Locale.ENGLISH);
        return resolver;
    }
}
```

```
1  spring.application.name=Resful_2
2  spring.messages.basename=messages
3  spring.web.locale-resolver=fixed
4  spring.web.locale=en
5  |
```

Q2)Content Negotiation

A) Create POST Method to create user details which can accept XML for user creation.



```
@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService userService; 3 usages

    public UserController(UserService userService) {
        this.userService = userService;
    }

    // A) POST - Accept XML to create user
    @PostMapping(consumes = { MediaType.APPLICATION_XML_VALUE, MediaType.APPLICATION_JSON_VALUE })
    public User createUser(@RequestBody User user) {
        return userService.createUser(user);
    }
}
```

```
import com.fasterxml.jackson.annotation.JsonPropertyOrder;
import com.fasterxml.jackson.dataformat.xml.annotation.JacksonXmlRootElement;

@JacksonXmlRootElement(localName = "User") 12 usages
@JsonPropertyOrder({ "id", "name"})
public class User {
    private Long id; 3 usages

    private String name; 3 usages

    public User(Long id, String name) { 3 usages
        this.id = id;
        this.name = name;
    }
}
```

B) Create GET Method to fetch the list of users in XML format.

HTTP / JVM / ResFul_Part_2 / New Request Save Share

GET Send http://localhost:8080/users

Params Authorization Headers (8) Body Scripts Settings Cookies

Headers 7 hidden

	Key	Value	Description	...	Bulk Edit	Presets
☐	Accept	application/json				
	Key	Value	Description			

Body Cookies Headers (5) Test Results 200 OK 3 ms 298 B Save Response

XML Preview Visualize

```

1 <List>
2   <item>
3     <id>1</id>
4     <name>Akash</name>
5   </item>
6   <item>
7     <id>2</id>
8     <name>Sahil</name>
9   </item>
10  <item>
11    <id>3</id>
12    <name>Aman</name>
13  </item>
14 </List>
```

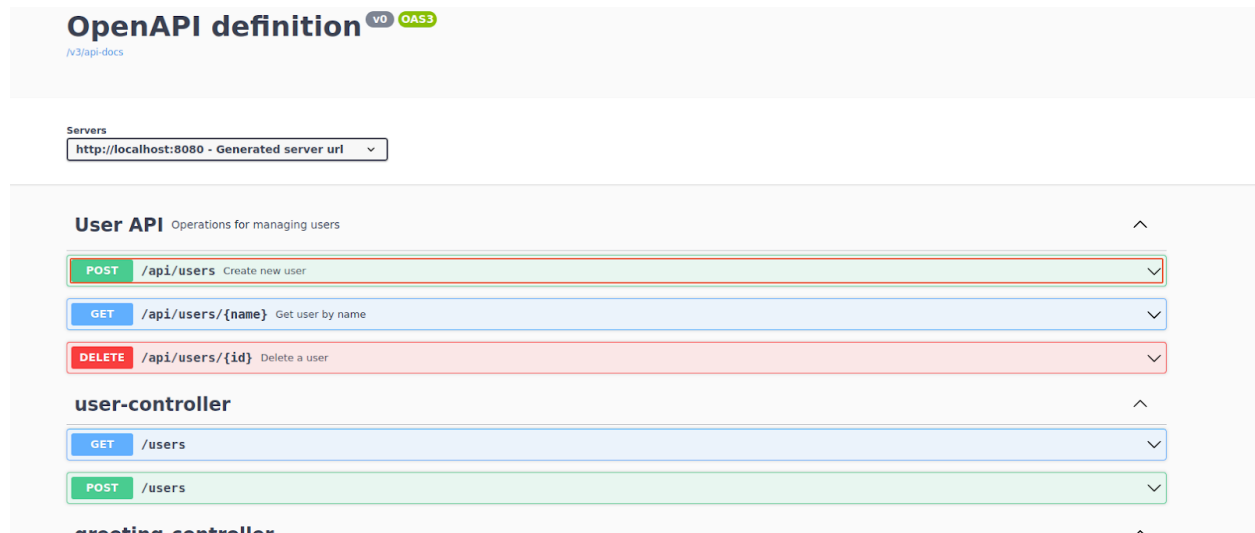
```
        return userService.createUser(user);
    }

    // B) GET - Return XML or JSON list of users
    @GetMapping(produces = { MediaType.APPLICATION_XML_VALUE, MediaType.APPLICATION_JSON_VALUE })
    public List<User> getAllUsers() {
        return userService.getAllUsers();
    }
}
```

Q3)Swagger

A) Configure the swagger plugin and create a document of the following methods: Get details of the User using GET request. Save details of the user using POST request. Delete a user using DELETE request.

B) In swagger documentation, add the description of each class and URI so that in swagger UI the purpose of class and URI is clear.



```
public class SwaggerUserController {

    private final UserService userService; 4 usages

    public SwaggerUserController(UserService userService) {
        this.userService = userService;
    }

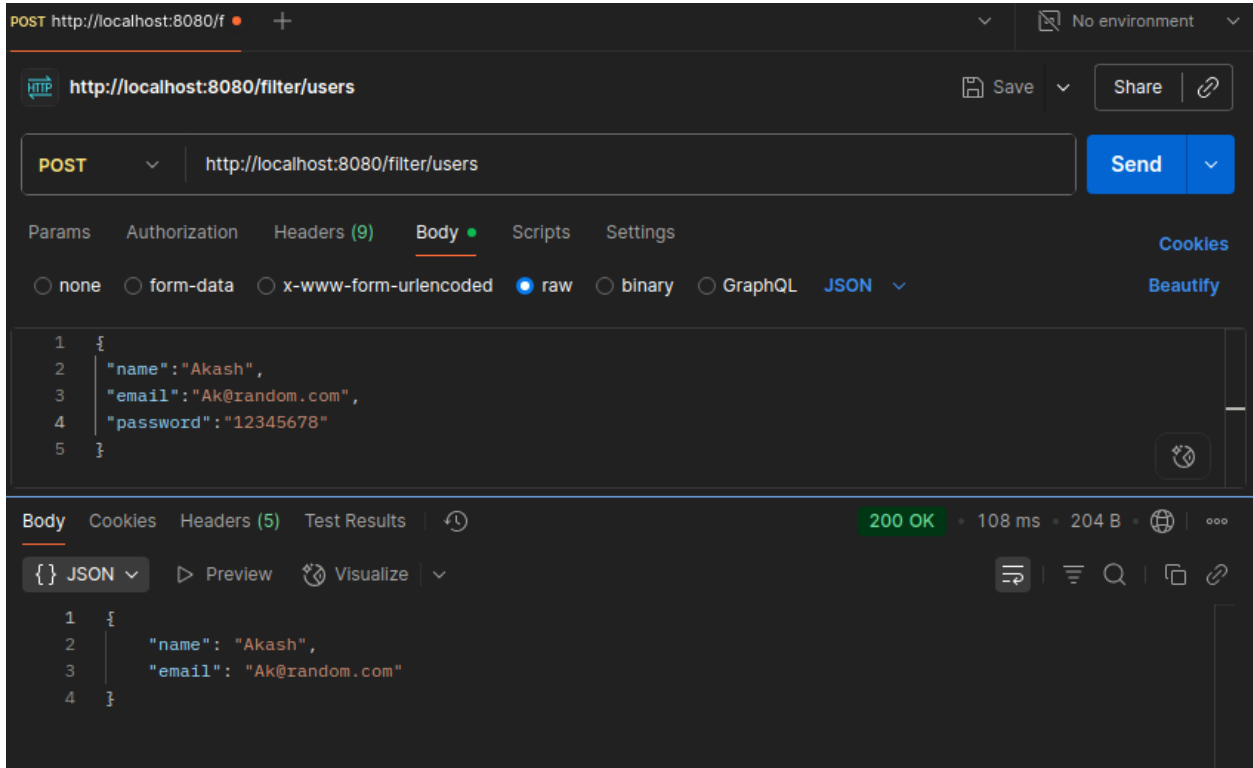
    @Operation(summary = "Get user by name")
    @GetMapping("/{name}")
    public User getUser(@PathVariable String name) {
        return userService.getAllUsers().get(0);
    }

    @Operation(summary = "Create new user")
    @PostMapping
    public User saveUser(@RequestBody User user) {
        return userService.createUser(user);
    }

    @Operation(summary = "Delete a user")
    @DeleteMapping("/{id}")
    public User deleteUser(@PathVariable int id) {
        return userService.deleteUserById(id);
    }
}
```


Q4) Static and Dynamic filtering

A) Create API which saves details of User (along with the password) but on successfully saving returns only non-critical data. (Use static filtering)



The screenshot shows a Postman interface for a POST request to `http://localhost:8080/filter/users`. The request body is a JSON object: `{ "name": "Akash", "email": "Ak@random.com", "password": "12345678" }`. The response is `200 OK` with a JSON body: `{ "name": "Akash", "email": "Ak@random.com" }`. The response body is displayed in the JSON view, showing that only the non-critical data (name and email) is returned.

```
@RestController
@RequestMapping("/filter")
public class FilterController {

    @PostMapping("/users")
    public UserEntity createUser(@RequestBody UserEntity user) {
        return user;
    }
}
```

```

public class UserEntity { no usages

    private String name; 3 usages
    private String email; 3 usages

    @JsonIgnore 3 usages
    private String password;

    public UserEntity(String name, String email, String password) { no usages
        this.name = name;
        this.email = email;
        this.password = password;
    }
}

```

B) Create another API that does the same by using Dynamic Filtering.

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/filter/dynamic/users`. The request body is a JSON object: `{ "name": "Akash", "email": "Ak@random.com", "password": "12345678" }`. The response is a `200 OK` status with a response time of 108 ms and a body size of 204 B. The response body is also shown as a JSON object: `{ "name": "Akash", "email": "Ak@random.com" }`.

```

@PostMapping("@"/dynamic/users")
public MappingJacksonValue createDynamicUser(@RequestBody UserEntity2 user) {
    MappingJacksonValue mappingJacksonValue = new MappingJacksonValue(user);

    SimpleBeanPropertyFilter propertyFilter = SimpleBeanPropertyFilter.filterOutAllExcept(...propertyArray: "name", "email");
    FilterProvider filterProvider = new SimpleFilterProvider().addFilter(id: "userFilter", propertyFilter);
    mappingJacksonValue.setFilters(filterProvider);
    return mappingJacksonValue;
}

```

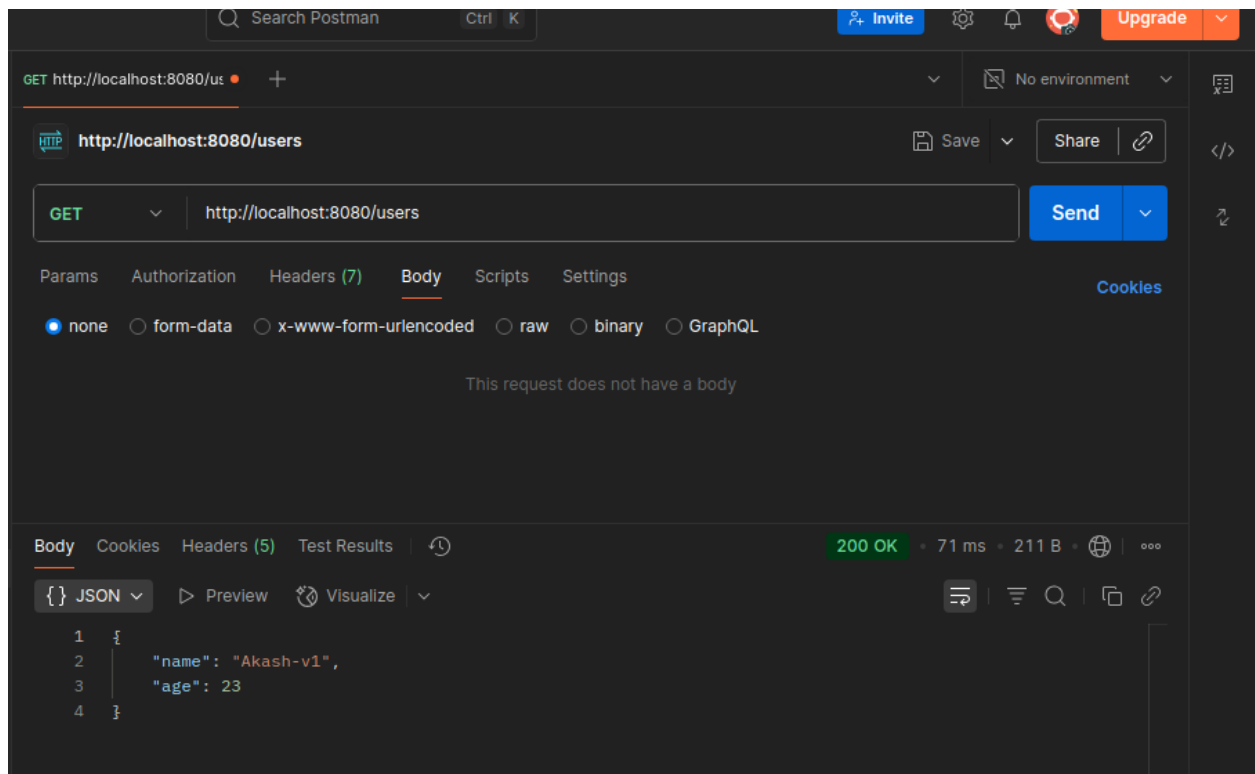
```
@JsonFilter("userFilter") 2 usages
public class UserEntity2 {

    private String name; 3 usages
    private String email; 3 usages
    private String password; 3 usages

    public UserEntity2(String name, String email, String password) { no usages
        this.name = name;
        this.email = email;
        this.password = password;
    }
}
```

Q5) Versioning Restful APIs Create 2 API for showing user details. The first api should return only basic details of the user and the other API should return more/enhanced details of the user, Now apply versioning using the following methods:

A) MimeType Versioning



HTTP <http://localhost:8080/users> Save Share

GET <http://localhost:8080/users> Send

Params Authorization **Headers (8)** Body Scripts Settings Cookies

☒	Accept	✚✚	
☒	Accept-Encoding	gzip, deflate, br	
☒	Connection	keep-alive	
☒	Accept	application/vnd.company.app-v2+json	
	Key	Value	Description

Body Cookies Headers (5) Test Results 200 OK 4 ms 231 B 🌐 ⋮

{} JSON Preview Visualize 🔍 📄 🔗

```
1 {
2   "firstName": "Akash",
3   "lastName": "Kumar",
4   "age": 23
5 }
```

```
import org.springframework.web.bind.annotation.RestController;

@RestController
public class VersionController {

    //MIME
    @GetMapping(value = "/users", produces = "application/vnd.company.app-v1+json")
    public UserV1 getUsers() {
        return new UserV1( name: "Akash-v1", age: 23);
    }

    @GetMapping(value = "/users", produces = "application/vnd.company.app-v2+json")
    public UserV2 getUsers2() {
        return new UserV2( firstName: "Akash", lastName: "Kumar", age: 23);
    }

}
```

B) Request Parameter versioning

HTTP <http://localhost:8080/users?version=1> Save Share

GET <http://localhost:8080/users?version=1> Send

Params Authorization Headers (8) Body Scripts Settings Cookies

Key	Value	Description
Accept	*/*	
Accept-Encoding	gzip, deflate, br	
Connection	keep-alive	
Accept	application/vnd.company.app-v2+json	

Body Cookies Headers (5) Test Results 200 OK • 3 ms • 189 B • 🌐 • ⋮

{ } JSON Preview Visualize

```
1 {
2   "name": "Akash",
3   "age": 23
4 }
```

HTTP <http://localhost:8080/users?version=2> Save Share

GET <http://localhost:8080/users?version=2> Send

Params Authorization Headers (8) Body Scripts Settings Cookies

Key	Value	Description
Accept	*/*	
Accept-Encoding	gzip, deflate, br	
Connection	keep-alive	
Accept	application/vnd.company.app-v2+json	

Body Cookies Headers (5) Test Results 200 OK • 6 ms • 212 B • 🌐 • ⋮

{ } JSON Preview Visualize

```
1 {
2   "firstName": "Akash",
3   "lastName": "Kumar",
4   "age": 23
5 }
```

```

@GetMapping(value = @RequestMapping("/users", params = "version=1")
public UserV1 getUserParamV1() { return new UserV1( name: "Akash", age: 23); }

@GetMapping(value = @RequestMapping("/users", params = "version=2")
public UserV2 getUserParamV2() { return new UserV2( firstName: "Akash", lastName: "Kumar", age: 23); }

```

C) URI versioning

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8080/v1/users`
- Method:** `GET`
- Headers (8):**

Key	Value	Description
Accept	*/*	
Accept-Encoding	gzip, deflate, br	
Connection	keep-alive	
Accept	application/vnd.company.app-v2+json	
Key	Value	Description
- Body:**

```

1 {
2   "name": "Akash-ur1",
3   "age": 23
4 }

```
- Status:** `200 OK`, 70 ms, 193 B

HTTP <http://localhost:8080/v2/users> Save Share

GET <http://localhost:8080/v2/users> Send

Params Authorization Headers (8) Body Scripts Settings Cookies

Key	Value	Description
☒ Accept	*/*	
☒ Accept-Encoding	gzip, deflate, br	
☒ Connection	keep-alive	
☐ Accept	application/vnd.company.app-v2+json	

Body Cookies Headers (5) Test Results 200 OK • 6 ms • 216 B

{ } JSON Preview Visualize

```
1 {
2   "firstName": "Akash-ur1",
3   "lastName": "Kumar",
4   "age": 23
5 }
```

//Url

```
@GetMapping(value = "/v1/users")
```

```
public UserV1 getUserUr1V1() { return new UserV1( name: "Akash-ur1", age: 23); }
```

```
@GetMapping(value = "/v2/users")
```

```
public UserV2 getUserUr1V2() { return new UserV2( firstName: "Akash-ur1", lastName: "Kumar", age: 23); }
```


D) Custom Header Versioning

HTTP <http://localhost:8080/users> Save Share

GET <http://localhost:8080/users> Send

Params Authorization **Headers (9)** Body Scripts Settings Cookies

Key	Value	Description
☒ Accept-Encoding	gzip, deflate, br	
☒ Connection	keep-alive	
☐ Accept	application/vnd.company.app-v2+json	
☒ API-VERSION	1	

Body Cookies Headers (5) Test Results 200 OK • 71 ms • 196 B • 🌐 ⋮

{} JSON Preview Visualize 🔍 📄 🔗

```
1 {  
2   "name": "Akash-header",  
3   "age": 23  
4 }
```

HTTP <http://localhost:8080/users> Save Share

GET <http://localhost:8080/users> Send

Params Authorization **Headers (9)** Body Scripts Settings Cookies

Key	Value	Description
☒ Accept-Encoding	gzip, deflate, br	
☒ Connection	keep-alive	
☐ Accept	application/vnd.company.app-v2+json	
☒ API-VERSION	2	

Body Cookies Headers (5) Test Results 200 OK • 7 ms • 219 B • 🌐 ⋮

{} JSON Preview Visualize 🔍 📄 🔗

```
1 {  
2   "firstName": "Akash-header",  
3   "lastName": "Kumar",  
4   "age": 23  
5 }
```

```
//headers
@GetMapping(value = @PathVariable("/users"),headers = "API-VERSION=1")
public UserV1 getUserHeaderV1() { return new UserV1( name: "Akash-header", age: 23); }

@GetMapping(value = @PathVariable("/users"),headers = "API-VERSION=2")
public UserV2 getUserHeaderV2() { return new UserV2( firstName: "Akash-header", lastName: "Kumar", age: 23); }
```

Q6)HATEOAS

A) Configure hateoas with your springboot application. Create an api which returns User Details along with url to show all topics.

The screenshot shows the Postman interface with a workspace titled "http://localhost:8080/hateos/users/1 - My Workspace". The left sidebar displays a collection of requests, including "GET New Request" and "ResFul_Part_2". The main panel shows a GET request to "http://localhost:8080/hateos/users/1". The response is a 200 OK status with a JSON body:

```
1 {
2   "id": 1,
3   "name": "Akash",
4   "_links": {
5     "all-users": {
6       "href": "http://localhost:8080/hateos/users"
7     }
8   }
9 }
```

The bottom status bar indicates "200 OK" and shows the response body in JSON format. The interface also includes a search bar, a "Search Postman" button, and a "Ctrl K" shortcut.

```
import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.*;

@RestController
@RequestMapping("/hateos")
public class HateosController {
    @Autowired
    private UserService userService;
    @GetMapping("/users")
    public List<User> getAllUsers() {
        return userService.getAllUsers();
    }
    @GetMapping("/users/{id}")
    public EntityModel<User> getUserById(@PathVariable int id) {
        User user = userService.findOne(id);
        EntityModel<User> entityModel = EntityModel.of(user);
        WebMvcLinkBuilder link = linkTo(methodOn(this.getClass()).getAllUsers());
        entityModel.add(link.withRel("all-users"));
        return entityModel;
    }
}
```